

**ĐẠI HỌC ĐÀ NẴNG**  
**TRƯỜNG ĐẠI HỌC BÁCH KHOA**  
**KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO**



**BÁO CÁO BÀI TẬP**

**Chuyên đề 2: Bài Tập 3**

|                      |                                                                           |
|----------------------|---------------------------------------------------------------------------|
| Sinh viên thực hiện: | <b>Đinh Hoàng Thuận - 106220271</b><br><b>Lê Văn Minh Trí - 106220238</b> |
| STT nhóm:            |                                                                           |
| GVHD:                | <b>TS. Nguyễn Văn Hiếu</b>                                                |
| Lớp học phần:        | <b>22.44</b>                                                              |
| Lớp sinh hoạt:       | <b>22KTMT</b>                                                             |

## 1. Lập lịch các nhiệm vụ với ưu tiên (Task Scheduling)

Lời giải bằng ngôn ngữ Python tại tệp:

<https://github.com/Backtodagame/Algorithm-Evaluation/blob/main/BT3/bt1.py>

## 2. Thuật toán Dijkstra

Khảo sát thuật toán Dijkstra và các cải tiến để tìm đường đi ngắn nhất giữa 2 nút trên đồ thị vô hướng.

- Giải thích thuật toán và các phương pháp cải tiến
- Phân tích độ phức tạp
- So sánh thực thi thời gian thực

Lời giải bằng ngôn ngữ C++ tại tệp:

[https://github.com/Backtodagame/Algorithm-Evaluation/blob/main/BT3/Dijkstra\\_BT3.2.ipynb](https://github.com/Backtodagame/Algorithm-Evaluation/blob/main/BT3/Dijkstra_BT3.2.ipynb)

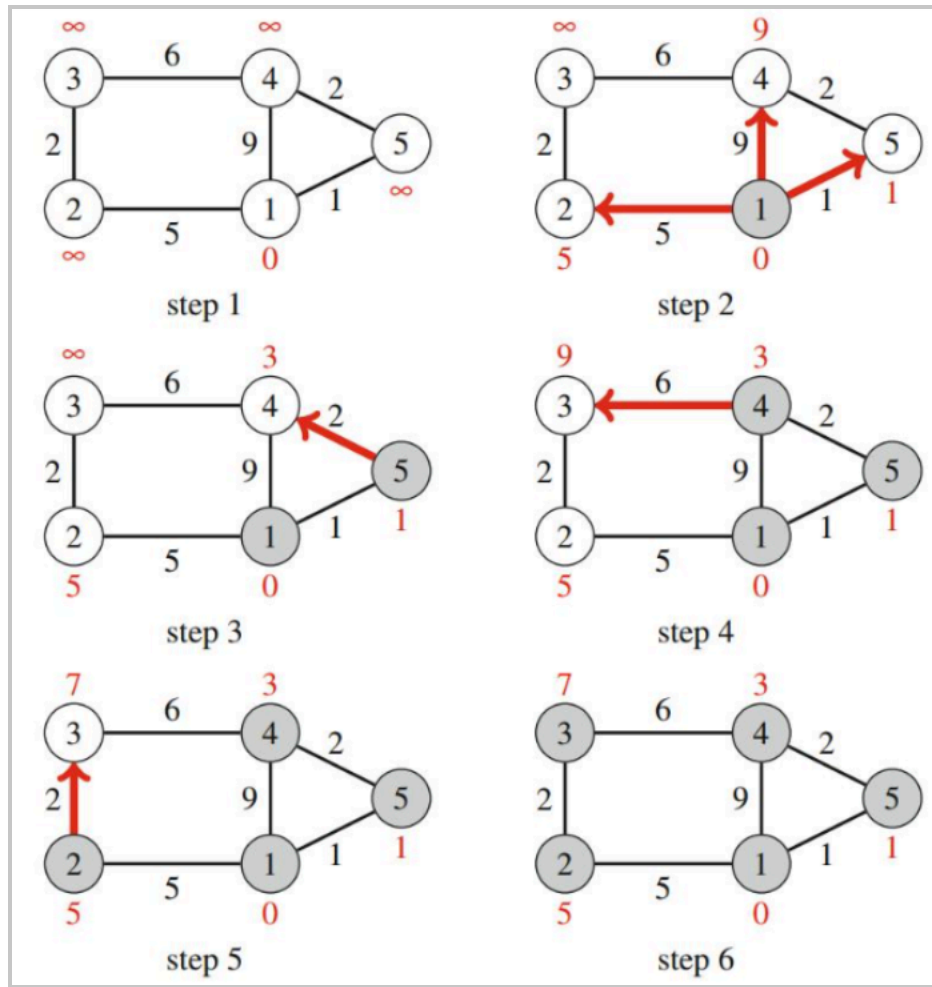
## 1. Mục tiêu bài toán:

- Khảo sát thuật toán Dijkstra: Cài đặt và so sánh hiệu năng giữa thuật toán Dijkstra truyền thống (sử dụng mảng) và bản cải tiến (sử dụng Min-Heap / Priority Queue) trên đồ thị vô hướng để tìm đường đi ngắn nhất giữa hai đỉnh.
- Phân tích độ phức tạp lý thuyết của từng cách cài đặt, đồng thời đo thời gian thực thi (mili-giây) trên nhiều kịch bản đồ thị khác nhau (đồ thị thưa và đồ thị dày) để đối chiếu với lý thuyết.
- Trực quan hóa kết quả đo được nhằm đưa ra đánh giá và kết luận về việc lựa chọn cấu trúc dữ liệu phù hợp khi cài đặt Dijkstra.

## 2. Cơ sở lý thuyết:

### 2.1. Thuật toán Dijkstra:

- Ý tưởng cốt lõi: Thuật toán Dijkstra tìm đường đi ngắn nhất từ đỉnh nguồn S đến các đỉnh còn lại trên đồ thị có trọng số không âm bằng chiến lược Tham ăn (Greedy).
- Phương pháp tiếp cận của thuật toán như sau:
  - Khởi tạo khoảng cách  $d[S] = 0$ , mọi đỉnh khác  $d[v] = \infty$ .
  - Lần lượt chọn đỉnh u chưa cố định có  $d[u]$  nhỏ nhất, đánh dấu u đã xử lý.
  - Tối ưu hóa (Relaxation): Với mỗi đỉnh kề v của u, nếu  $d[v] > d[u] + w(u, v)$  thì cập nhật  $d[v] = d[u] + w(u, v)$ .



**Hình 1: Minh họa các bước Dijkstra**

- Dựa vào hình minh họa, ta có các bước sau đây:
- **Khởi tạo:** Đặt khoảng cách từ đỉnh nguồn đến chính nó bằng 0, các đỉnh còn lại bằng vô cùng.
- **Chọn đỉnh:** Trong các đỉnh chưa được xử lý, chọn đỉnh có khoảng cách tạm thời nhỏ nhất để xử lý.
- **Relaxation:** Xét các đỉnh kề của đỉnh vừa chọn. Nếu tìm được đường đi mới có tổng trọng số nhỏ hơn khoảng cách hiện tại thì cập nhật khoảng cách và đỉnh cha.
- **Đánh dấu đỉnh:** Sau khi xử lý xong, đỉnh được chọn được đánh dấu là đã cố định và không xét lại khoảng cách của nó.
- **Lặp lại:** Tiếp tục chọn đỉnh chưa xử lý có khoảng cách nhỏ nhất và thực hiện quá trình trên cho đến khi đạt đỉnh đích hoặc không còn đỉnh nào có thể cập nhật.

- **Kết quả:** Sau khi thuật toán kết thúc, các giá trị khoảng cách được cập nhật là khoảng cách ngắn nhất từ đỉnh nguồn đến các đỉnh tương ứng.

## 2.2. Các cách tiếp cận cho thuật toán Dijkstra:

- Trong bài tập, đồ thị vô hướng được lưu bằng danh sách kề. Hai phiên bản Dijkstra dùng chung bước khởi tạo khoảng cách, phép nối lỏng cạnh và điều kiện dừng sớm khi lấy được đỉnh đích; điểm khác nhau chủ yếu nằm ở cấu trúc dùng để chọn đỉnh có khoảng cách tạm thời nhỏ nhất.

### 2.2.1. Sử dụng Mảng:

- **Nguyên lý:** Sử dụng mảng  $dist$  để lưu khoảng cách ngắn nhất tạm thời từ đỉnh nguồn và mảng  $visited$  để đánh dấu các đỉnh đã được cố định.
  - Khởi tạo  $dist[start] = 0$ , các phần tử còn lại bằng vô cùng và toàn bộ  $visited$  bằng  $false$ .
  - Ở mỗi vòng lặp, duyệt toàn bộ  $V$  đỉnh để tìm đỉnh  $u$  chưa xử lý có  $dist[u]$  nhỏ nhất. Nếu không còn đỉnh hợp lệ thì kết thúc; nếu  $u$  là đỉnh đích thì dừng sớm.
  - Đánh dấu  $u$  đã xử lý, sau đó duyệt các cạnh kề  $(u, v)$ . Nếu  $dist[v] > dist[u] + w(u, v)$  thì cập nhật khoảng cách của  $v$ .
- **Độ phức tạp:** Bước chọn đỉnh nhỏ nhất được thực hiện tối đa  $V$  lần và mỗi lần quét  $V$  phần tử, nên thời gian là  $O(V^2 + E)$ , thường viết gọn là  $O(V^2)$ . Với cách lưu đồ thị bằng danh sách kề, bộ nhớ sử dụng là  $O(V + E)$ .
- **Đánh giá:** Cách cài đặt này đơn giản, dễ kiểm tra và không phát sinh chi phí quản lý heap. Tuy nhiên, thao tác quét toàn bộ mảng ở mỗi vòng lặp làm thời gian tăng nhanh khi số đỉnh lớn; phương án phù hợp hơn với đồ thị nhỏ hoặc đồ thị rất dày.

### 2.2.2. Sử dụng Min-Heap:

- **Nguyên lý:** Thay phép quét mảng bằng hàng đợi ưu tiên  $priority\_queue$  được cấu hình như Min-Heap. Mỗi phần tử lưu cặp (khoảng cách, đỉnh), do đó phần tử có khoảng cách nhỏ nhất luôn nằm ở đỉnh heap.
  - Khởi tạo  $dist[start] = 0$  và đưa cặp  $(0, start)$  vào heap.
  - Lấy phần tử  $(d, u)$  nhỏ nhất ra khỏi heap. Nếu  $d > dist[u]$  thì đây là bản ghi cũ và được bỏ qua; nếu  $u$  là đỉnh đích thì dừng sớm.

- Duyệt các đỉnh kề  $v$  của  $u$ . Mỗi khi tìm được khoảng cách tốt hơn, cập nhật  $\text{dist}[v]$  và đưa cặp mới  $(\text{dist}[v], v)$  vào heap.
- **Độ phức tạp:** Mỗi lần thêm hoặc lấy phần tử khỏi heap có chi phí  $O(\log V)$ , nên tổng thời gian là  $O((V + E)\log V)$ , thường biểu diễn là  $O(E \log V)$  đối với đồ thị liên thông. Bộ nhớ là  $O(V + E)$ ; heap có thể chứa đồng thời một số bản ghi cũ chưa được loại bỏ.
- **Đánh giá:** Min-Heap tránh việc quét toàn bộ tập đỉnh và thường hiệu quả rõ rệt trên đồ thị lớn, thưa. Đổi lại, cấu trúc heap phức tạp hơn và có chi phí thêm cho các thao tác đẩy, lấy phần tử cũng như kiểm tra bản ghi cũ.

### 3. Phương pháp đánh giá:

- **Mục tiêu đánh giá:** So sánh thời gian thực thi của hai hàm `dijkstraArray` và `dijkstraHeap` khi số đỉnh hoặc số cạnh của đồ thị thay đổi, đồng thời đối chiếu xu hướng thực nghiệm với độ phức tạp lý thuyết.
- **Dữ liệu thử nghiệm:** Đồ thị vô hướng được sinh tự động bằng hàm `generateGraph(V, E)`. Bộ sinh số giả ngẫu nhiên dùng hạt giống 42 để có thể tái lập dữ liệu; hai đầu mút của cạnh được chọn trong  $[0, V - 1]$ , trọng số nguyên dương được chọn trong  $[1, 100]$ . Cạnh tự khuyên bị loại bỏ, còn các cạnh song song có thể xuất hiện.
- **Kịch bản khảo sát:** Chương trình thực hiện hai nhóm cấu hình nhằm quan sát riêng ảnh hưởng của  $V$  và  $E$ .
  - Kịch bản đồ thị thưa:  $E = 4V$ ;  $V$  nhận các giá trị 1.000, 3.000, ..., 19.000.
  - Kịch bản thay đổi mật độ cạnh (nhãn `Dense_Graph`): cố định  $V = 3.000$  và lần lượt đặt  $E$  bằng 10.000, 50.000, 200.000, 1.000.000 và 3.000.000.
- **Quy trình đo:** Với mỗi cấu hình, chỉ sinh một đồ thị và dùng chính đồ thị đó cho cả hai phiên bản. Đỉnh nguồn được cố định là 0, đỉnh đích là  $V - 1$ . Thời điểm bắt đầu và kết thúc mỗi lần chạy được ghi bằng `chrono::high_resolution_clock`, sau đó quy đổi thời gian sang mili-giây.
- **Lưu trữ và trực quan hóa:** Mỗi phép đo được ghi vào tệp `dijkstra_benchmark.csv` với các trường `Scenario`,  $V$ ,  $E$ , `Time_Array_ms` và `Time_Heap_ms`.
- **Tiêu chí so sánh:** Chỉ số chính là thời gian chạy (ms), được quan sát theo  $V$  hoặc  $E$ . Phiên bản có thời gian thấp hơn và tốc độ tăng chậm hơn khi kích thước đồ thị tăng được xem là hiệu quả hơn trong kịch bản tương ứng.
- **Giới hạn của phép đo:** Mỗi cấu hình hiện được chạy một lần nên kết quả có thể chịu ảnh hưởng của tải hệ thống và cơ chế tối ưu của trình biên dịch. Ngoài

ra, hàm sinh đồ thị không bảo đảm liên thông; việc bỏ qua cạnh tự khuyên làm số cạnh thực tế có thể nhỏ hơn E, trong khi cạnh song song vẫn được giữ. Vì vậy, khi cần độ tin cậy thống kê cao hơn nên chạy lặp nhiều lần và báo cáo giá trị trung bình hoặc trung vị.

## 4. Kết quả:

Kết quả đo thời gian thực thi (ms) của hai phiên bản dijkstraArray và dijkstraHeap trên tệp dijkstra\_benchmark.csv được tổng hợp lại như sau:

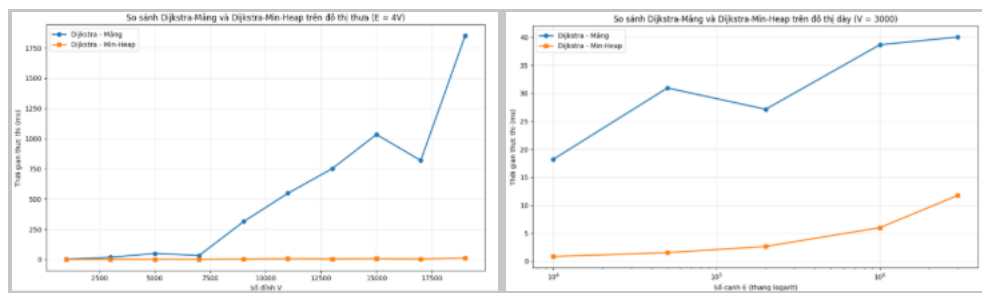
**Bảng 1: Kích bản đồ thị thưa ( $E = 4V$ )**

| V      | E      | Time_Array (ms) | Time_Heap (ms) |
|--------|--------|-----------------|----------------|
| 1.000  | 4.000  | 2,73262         | 0,315164       |
| 3.000  | 12.000 | 17,9085         | 0,832414       |
| 5.000  | 20.000 | 50,0765         | 1,19765        |
| 7.000  | 28.000 | 33,4839         | 0,619801       |
| 9.000  | 36.000 | 313,259         | 3,23771        |
| 11.000 | 44.000 | 547,883         | 5,97808        |
| 13.000 | 52.000 | 751,339         | 5,21997        |
| 15.000 | 60.000 | 1.034,49        | 6,14053        |
| 17.000 | 68.000 | 819,195         | 4,26415        |
| 19.000 | 76.000 | 1.850,47        | 11,8023        |

**Bảng 2: Kích bản đồ thị dày ( $V = 3.000$  cố định, E tăng dần)**

| V     | E         | Time_Array (ms) | Time_Heap (ms) |
|-------|-----------|-----------------|----------------|
| 3.000 | 10.000    | 18,2179         | 0,862155       |
| 3.000 | 50.000    | 30,9535         | 1,54023        |
| 3.000 | 200.000   | 27,1399         | 2,65707        |
| 3.000 | 1.000.000 | 38,6574         | 6,00331        |
| 3.000 | 3.000.000 | 40,0378         | 11,7672        |

Biểu đồ trực quan hóa minh họa hai kịch bản trên:



**Hình 2: So sánh thời gian thực thi giữa Dijkstra-Mảng và Dijkstra-Min-Heap trên đồ thị thưa (trái) và đồ thị dày (phải, trục E theo thang logarit).**

## 5. Nhận xét:

- Trên đồ thị thưa ( $E = 4V$ ): thời gian chạy của bản Mảng nhìn chung tăng mạnh khi số đỉnh  $V$  tăng, từ 2,73262 ms tại  $V = 1.000$  lên 1.850,47 ms tại  $V = 19.000$ . Tuy nhiên, thời gian không tăng hoàn toàn đơn điệu, chẳng hạn tại  $V = 7.000$  thời gian là 33,4839 ms, thấp hơn 50,0765 ms tại  $V = 5.000$ ; tương tự  $V = 17.000$  có thời gian 819,195 ms, thấp hơn 1.034,49 ms tại  $V = 15.000$ . Dù có dao động, xu hướng tổng thể của bản Mảng vẫn tăng nhanh theo  $V$ , phù hợp với độ phức tạp lý thuyết  $O(V^2)$ . Trong khi đó, bản Min-Heap tăng chậm hơn, từ 0,315164 ms lên 11,8023 ms, phù hợp với độ phức tạp  $O(E \log V)$ ; do  $E = 4V$  nên  $E$  tăng tuyến tính theo  $V$ .
- Sự dao động của thời gian đo có thể xuất hiện do mỗi cấu hình chỉ được chạy một lần, nên kết quả chịu ảnh hưởng bởi tải hệ thống và cơ chế tối ưu của trình biên dịch. Vì vậy, các điểm giảm thời gian ở một số giá trị  $V$  không có nghĩa là thuật toán có độ phức tạp giảm, mà chủ yếu phản ánh nhiễu của phép đo. Để có kết quả đáng tin cậy hơn, nên thực hiện nhiều lần đo và sử dụng giá trị trung bình hoặc trung vị. Điều này cũng phù hợp với giới hạn phép đo đã nêu trong báo cáo.
- Trên đồ thị dày ( $V = 3.000$  cố định,  $E$  tăng): thời gian của bản Mảng tăng từ 18,2179 ms khi  $E = 10.000$  lên 40,0378 ms khi  $E = 3.000.000$ . Mặc dù có một điểm giảm nhẹ từ 30,9535 ms xuống 27,1399 ms, xu hướng chung vẫn là tăng. Do  $V$  được giữ cố định, thành phần  $O(V^2)$  của bản Mảng không thay đổi theo  $V$ ; tuy nhiên thời gian thực tế vẫn chịu ảnh hưởng của việc duyệt các cạnh trong danh sách kề. Trong khi đó, bản Min-Heap tăng từ 0,862155 ms lên 11,7672 ms khi  $E$  tăng, thể hiện rõ sự phụ thuộc vào số cạnh theo thành phần  $O(E \log V)$ .
- Kết quả thực nghiệm cho thấy Min-Heap nhanh hơn bản Mảng ở tất cả các cấu hình đã thử nghiệm. Trên đồ thị thưa, ưu thế của Min-Heap càng rõ khi số đỉnh tăng; tại  $V = 19.000$ , thời gian của bản Mảng là 1.850,47 ms trong khi Min-Heap chỉ mất 11,8023 ms, tức nhanh hơn khoảng 156,8 lần. Trên đồ thị dày, khi số cạnh tăng, thời gian của Min-Heap tăng rõ rệt và khoảng cách giữa hai phương pháp có xu hướng thu hẹp, nhưng Min-Heap vẫn có thời gian thực thi thấp hơn bản Mảng trong toàn bộ các trường hợp đo được. Kết quả này phù hợp với nhận định lý thuyết rằng Min-Heap đặc biệt hiệu quả đối với đồ thị lớn và thưa.

